

DocuMind AI

Complete File-by-File Developer Documentation

Purpose of this document:

This document is a developer reference for the complete DocuMind AI project. It explains every major source, test, evaluation, configuration, and data-related file created during the project, including its purpose, internal responsibilities, dependencies, interactions, design decisions, and important revision points.

1. Project Overview

DocuMind AI is an intelligent document analysis platform built around:

- Document ingestion
- PDF processing
- OCR
- Text preprocessing
- Page-aware chunking
- Gemini embeddings
- FAISS vector search
- Retrieval-Augmented Generation
- Source/page citations
- Structured information extraction
- Tool-calling agents
- Multi-document comparison
- FastAPI
- Streamlit
- Automated testing
- Evaluation

The architecture intentionally separates these responsibilities into different modules.

The main design principle is:

```
Raw Documents
  ↓
Document Processing
  ↓
Chunking
  ↓
Embeddings
  ↓
Vector Store
  ↓
Retrieval
```

↓
LLM
↓
Answer / Extraction / Agent / Comparison
↓
API
↓
Streamlit

2. Complete Project Structure

```
documind-ai/
├── app/
│   ├── api/
│   │   └── main.py
│   ├── document/
│   │   ├── loader.py
│   │   ├── ocr.py
│   │   └── preprocessing.py
│   ├── rag/
│   │   ├── chunker.py
│   │   ├── embeddings.py
│   │   ├── vector_store.py
│   │   ├── retriever.py
│   │   └── pipeline.py
│   ├── agents/
│   │   ├── tools.py
│   │   └── document_agent.py
│   ├── services/
│   │   ├── extraction_service.py
│   │   └── comparison_service.py
│   ├── models/
│   │   └── schemas.py
│   └── core/
│       ├── config.py
│       └── logger.py
└── frontend/
    └── app.py
```

```

├── tests/
│   ├── test_api.py
│   ├── test_rag.py
│   ├── test_services.py
│   ├── test_document.py
│   ├── test_agent.py
│   └── test_evaluator.py
├── evaluation/
│   ├── questions.json
│   └── evaluator.py
├── scripts/
├── data/
│   ├── raw/
│   │   ├── invoices/
│   │   ├── forms/
│   │   ├── contracts/
│   │   └── synthetic/
│   ├── processed/
│   └── vector_store/
├── requirements.txt
├── .env
├── .gitignore
└── README.md

```

3. How the Folders Are Organized

Before studying individual files, remember the responsibility of each folder.

Folder	Responsibility
<code>app/api</code>	Exposes application functionality through HTTP
<code>app/document</code>	Loads and prepares documents
<code>app/rag</code>	Implements retrieval-augmented generation
<code>app/agents</code>	Implements tool-calling agent behavior
<code>app/services</code>	Contains higher-level business logic
<code>app/models</code>	Defines shared data structures
<code>app/core</code>	Configuration and infrastructure utilities
<code>frontend</code>	Streamlit UI

Folder	Responsibility
<code>tests</code>	Automated tests
<code>evaluation</code>	Quality/evaluation framework
<code>data/raw</code>	Original documents
<code>data/processed</code>	Processed document data
<code>data/vector_store</code>	FAISS index
<code>scripts</code>	Utility/operational scripts

The architecture follows a rough separation:

```

core
↓
models
↓
document
↓
rag
↓
services / agents
↓
api
↓
frontend

```

4. `app/core/`

The `core` package contains infrastructure that should be shared across the application.

It does not contain document-specific business logic.

5. `app/core/config.py`

Purpose

`config.py` is the **central configuration layer** of DocuMind AI.

Instead of having individual modules read environment variables independently, this file provides one central source of configuration.

This became particularly important when the project migrated from OpenAI to Gemini.

Main Responsibilities

It manages configuration such as:

- Application information
- API host
- API port
- LLM provider
- LLM model
- LLM temperature
- Embedding provider
- Embedding model
- Gemini API key
- Chunk size
- Chunk overlap
- Retrieval `top_k`
- Data directories

Important Concept

Other modules should conceptually do:

```
settings.GEMINI_API_KEY
```

rather than repeatedly doing:

```
os.getenv("GEMINI_API_KEY")
```

throughout the application.

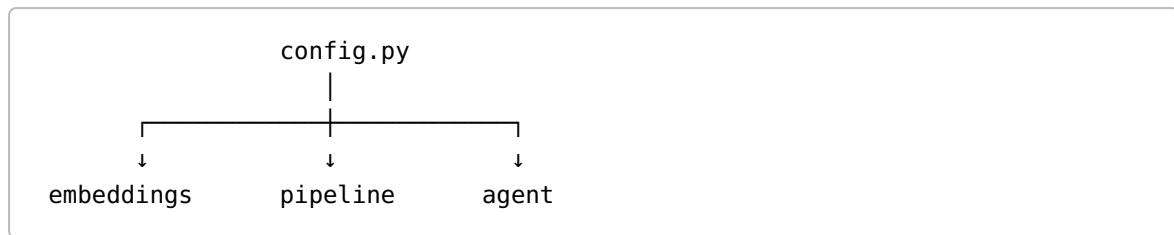
Why This File Exists

Without centralized configuration, we could end up with:

```
embeddings.py    → reads GEMINI_API_KEY  
pipeline.py      → reads GEMINI_API_KEY  
agent.py         → reads GEMINI_API_KEY  
extraction.py    → reads GEMINI_API_KEY
```

That creates duplicated configuration logic.

Instead:



Key Design Decision

Provider configuration was centralized so that the project could move from:

OpenAI

to:

Gemini

without redesigning the entire application.

Important Revision Point

If you change:

- Gemini model
- Embedding model
- Chunk size
- Retrieval count
- API port

this is one of the first files to inspect.

6. `app/core/logger.py`

Purpose

Provides centralized logging functionality.

Main Responsibility

Instead of configuring logging independently in every file, this module provides a reusable logger.

Conceptually:

```
logger = get_logger(__name__)
```

Why It Matters

Logging is useful for tracing:

```
Upload
↓
Loading
↓
Chunking
↓
Embedding
↓
FAISS
↓
Retrieval
↓
LLM
```

When something fails, logs tell you which stage failed.

Example Debugging Scenario

Our Gemini embedding problem demonstrated this.

The upload endpoint returned:

```
Failed to generate embeddings and index document
```

The detailed terminal traceback revealed:

```
text-embedding-004
404 NOT_FOUND
```

Without proper logging/error visibility, this would have been much harder to diagnose.

7. `app/models/schemas.py`

Purpose

Defines the application's shared Pydantic models.

This file represents the **data contracts** used throughout DocuMind.

Important Models

`DocumentMetadata`

Represents document-level metadata.

Typical information includes:

- Document ID
 - Filename
 - Document type
 - Other document metadata
-

`PageContent`

Represents one page of a processed document.

Conceptually:

Page 1
└─ text

Page 2
└─ text

`ProcessedDocument`

Represents a fully loaded document after processing.

It connects:

Document metadata
+

Pages
+
Full document content

ExtractedField

Represents one structured field extracted from a document.

Conceptually:

Field name
Value
Confidence
Source

SourceCitation

Represents the source location used to support an answer.

Important information includes document/page context.

ChatRequest

Represents a request to the RAG chat endpoint.

ChatResponse

Represents the generated answer and supporting information.

ComparisonField

Represents a field being compared between documents.

ComparisonResult

Represents the final document comparison result.

Why This File Is Important

This file acts as a shared contract.

For example:

```
FastAPI
  ↓
schemas
  ↓
service
  ↓
schemas
  ↓
frontend
```

This prevents every module from inventing its own data format.

8. `app/document/`

This package handles everything related to converting raw files into usable document content.

It is deliberately separated from RAG.

The document package answers:

"How do I turn this PDF/image into structured text?"

The RAG package answers:

"How do I search and reason over that text?"

9. `app/document/loader.py`

Purpose

Loads uploaded documents and converts them into the internal `ProcessedDocument` representation.

Main Responsibilities

- Detect/process PDFs
- Process images
- Generate document IDs

- Extract page text
 - Build page objects
 - Build document metadata
-

PDF Flow

```
PDF
↓
PyMuPDF
↓
Page 1
Page 2
Page 3
...
↓
ProcessedDocument
```

Why Page-by-Page?

Suppose the document contains:

```
Page 1 → Parties
Page 2 → Effective Date
Page 5 → Termination
```

If we combine everything without page metadata, later we cannot reliably say:

```
"The answer came from Page 2."
```

Therefore the loader preserves page boundaries.

Dependencies

Main dependencies include:

- PyMuPDF
 - Pillow
 - Pydantic schemas
 - Logger
-

Used By

Primarily:

```
main.py
```

and document processing workflows.

Critical Revision Point

If PDF parsing stops working, inspect this file first.

If extracted text looks wrong:

```
loader.py  
↓  
preprocessing.py
```

should be investigated before changing the RAG system.

10. app/document/ocr.py

Purpose

Provides OCR functionality.

Problem It Solves

Some PDFs contain actual text:

```
PDF  
↓  
Text extraction
```

Others are effectively images:

```
PDF  
↓  
Image
```

↓
No usable text

OCR solves the second problem:

Image
↓
Tesseract
↓
Text

Technologies

- Pillow
- pytesseract
- Tesseract OCR

Important Distinction

Installing:

```
pip install pytesseract
```

does not install the actual Tesseract executable.

Both components are required:

```
Python  
↓  
pytesseract  
↓  
Tesseract executable
```

Importance

Without this module, scanned/image-based documents would have limited usefulness.

11. `app/document/preprocessing.py`

Purpose

Cleans extracted text before it enters the RAG pipeline.

Why Preprocessing?

Raw extracted PDF text may contain:

- Excess whitespace
- Empty lines
- Formatting noise
- Extraction artifacts

Preprocessing tries to improve retrieval quality.

Important Design Decision

We intentionally kept preprocessing **conservative**.

For example, tables may rely on spacing.

Aggressively converting:

```
A      B      C
```

into:

```
A B C
```

could destroy useful structure.

Therefore preprocessing should clean noise without destroying document semantics.

Flow

```
Raw text
↓
Clean text
↓
PageContent
```

↓
Chunker

12. app/rag/

This is the core GenAI retrieval subsystem.

The modules work together:

```
chunker.py
  ↓
embeddings.py
  ↓
vector_store.py
  ↓
retriever.py
  ↓
pipeline.py
```

13. app/rag/chunker.py

Purpose

Splits documents into smaller chunks suitable for embedding and retrieval.

Why Chunk?

LLMs and embedding models should not receive an entire large document as one retrieval unit.

Instead:

```
Large Document
  ↓
Small chunks
  ↓
Embeddings
  ↓
Search
```

Main Technology

LangChain's recursive text splitting functionality.

Important Design

Chunking is performed **page-by-page**.

That means:

Page 1

↓

chunks

Page 2

↓

chunks

Page 3

↓

chunks

instead of:

Entire document

↓

chunks

Metadata

Each chunk retains information such as:

document_id

filename

page_number

This becomes essential for citations.

Settings

Chunking uses centralized configuration for:

```
CHUNK_SIZE
CHUNK_OVERLAP
```

Importance

This file controls one of the most important RAG quality factors:

How information is divided before retrieval.

Bad chunking can produce bad retrieval even when the LLM itself is excellent.

14. `app/rag/embeddings.py`

Purpose

Converts text into numerical vectors.

Concept

Text:

```
"The agreement is effective from January 1."
```

becomes something like:

```
[0.013, -0.082, 0.221, ...]
```

The vector represents semantic meaning.

Current Provider

Gemini.

Current Working Model

```
gemini-embedding-001
```

Main Responsibilities

- Initialize Gemini embeddings
 - Validate configuration
 - Embed documents
 - Embed queries
 - Handle embedding errors
-

Important Debugging History

Initially, the project attempted:

```
text-embedding-004
```

This produced:

```
404 NOT_FOUND
```

The API was queried for available embedding models.

It returned:

```
models/gemini-embedding-001  
models/gemini-embedding-2-preview  
models/gemini-embedding-2
```

We selected:

```
gemini-embedding-001
```

After changing the configuration, `/documents/upload` successfully indexed the document.

Importance

This is the bridge between:

```
Text
```

and:

15. `app/rag/vector_store.py`

Purpose

Manages FAISS.

What FAISS Does

FAISS stores embeddings and allows similarity search.

Conceptually:

```
Document chunks
  ↓
Embeddings
  ↓
FAISS index
```

When the user asks:

```
"What is the effective date?"
```

the query becomes a vector and FAISS searches for nearby document vectors.

Responsibilities

- Create FAISS store
 - Load FAISS store
 - Add documents
 - Persist store
 - Rebuild store
 - Return store
-

Important Security Consideration

The implementation uses:

```
allow_dangerous_deserialization=True
```

for trusted local vector-store data.

This should **not** be interpreted as:

"All FAISS files are safe."

It is appropriate only when the vector store is trusted.

Importance

This file provides persistent semantic search.

16. `app/rag/retriever.py`

Purpose

Retrieves relevant chunks from FAISS.

Input

User question

Output

Relevant document chunks

Main Responsibilities

- Similarity retrieval
 - `top_k`
 - Query validation
 - Document filtering
 - Retrieval with scores
-

Document Filtering

If multiple documents are indexed:

```
Document A
Document B
Document C
```

the retriever can restrict the search to selected document IDs.

This is important for:

```
"What does Document B say about termination?"
```

Important Score Concept

FAISS L2 distance is:

```
LOWER = MORE SIMILAR
HIGHER = LESS SIMILAR
```

It is **not** automatically:

```
0.95 = 95% confidence
```

This is an important revision point.

17. app/rag/pipeline.py

Purpose

This is the central RAG pipeline.

It combines:

```
Retriever
+
Prompt
+
Gemini LLM
```

+
Citation handling

Full Process

```
Question
↓
Retriever
↓
Relevant chunks
↓
Context
↓
Grounded prompt
↓
Gemini
↓
Answer
↓
Citations
```

Context Construction

Retrieved chunks are represented with source information similar to:

```
[Document: filename | Page: 3]
```

This allows the model to associate information with a source.

Citation Handling

The pipeline creates `SourceCitation` objects and removes duplicate citations.

Grounding

The system prompt encourages the LLM to answer from retrieved information rather than freely hallucinating.

Importance

If you want to understand the core of DocuMind AI's RAG functionality, this is one of the most important files to study.

18. `app/agents/`

This package provides the agentic layer.

The distinction is:

```
RAG
↓
Fixed retrieval workflow
```

versus:

```
Agent
↓
LLM decides which tool/capability to use
```

19. `app/agents/tools.py`

Purpose

Defines tools that the agent can invoke.

Main Tools

The implementation contains document capabilities including:

```
document_question_answering
extract_document_fields
```

`document_question_answering`

Allows the agent to ask questions against document content using the RAG system.

`extract_document_fields`

Allows the agent to perform structured extraction.

Helper Logic

The module also contains logic for:

- Parsing document text
 - Handling page markers
 - Building `ProcessedDocument`
 - Connecting tools to services
-

Important Concept

The tools are ordinary application functions exposed to the LLM in a controlled manner.

The LLM does not directly execute arbitrary Python code.

Importance

This file is the bridge between:

Agent

and:

Application capabilities

20. `app/agents/document_agent.py`

Purpose

Implements the document tool-calling agent.

Agent Flow

```
User Query
  ↓
Gemini
  ↓
Tool Selection
  ↓
Tool Execution
  ↓
Tool Result
  ↓
Gemini
  ↓
Final Answer
```

Example

User:

Find the effective date of this agreement and identify the parties involved.

The agent can use the available document tools to obtain the required information.

Why Not Just Use RAG?

RAG is ideal for:

```
"Where is the termination clause?"
```

An agent becomes useful when the request may require different capabilities.

For example:

```
"Extract the effective date and then compare it with another document."
```

The agent can coordinate capabilities.

Important Protection

The agent uses an iteration limit.

This protects against an uncontrolled loop such as:

```
Tool
↓
Tool
↓
Tool
↓
Tool
...
```

21. `app/services/`

The service layer contains business logic that is more complex than simple API handling.

22. `app/services/extraction_service.py`

Purpose

Provides structured document extraction.

Problem

RAG normally returns natural language.

Example:

```
"The effective date is January 1, 2025."
```

But applications often need:

```
effective_date = "January 1, 2025"
```

The extraction service addresses this.

Workflow

```
Document
↓
Context
↓
Gemini
↓
Structured output
↓
Pydantic validation
↓
ExtractionResult
```

Output

The extraction system uses models such as:

```
ExtractedField
```

with information such as:

- Field name
- Value
- Confidence
- Source

Important Validation

Confidence should be:

```
0.0 <= confidence <= 1.0
```

Importance

This module converts GenAI output into application-friendly structured data.

23. `app/services/comparison_service.py`

Purpose

Compares two documents.

Why Is It a Separate Service?

Comparison is more than retrieval.

It involves:

```
Extract
↓
Normalize
↓
Compare
↓
Summarize
```

Therefore it belongs in the service layer.

Normalization

Values can be normalized before comparison.

For example, differences in representation may be handled through normalization and synonym mappings.

Comparison

Conceptually:

Field	Document A	Document B
Effective Date	Jan 1	Jan 5
Payment Terms	30 days	45 days
Termination	30 days	60 days

The system can identify differences.

Important Edge Case

Numeric normalization can potentially cause:

```
00123
```

and:

```
123
```

to match.

That may be wrong for identifiers where leading zeros are meaningful.

This should be addressed if comparison becomes production-grade.

Importance

This is one of the project's strongest differentiating features.

24. `app/api/main.py`

Purpose

This is the main FastAPI application.

It is the **entry point of the backend**.

Responsibilities

It connects HTTP requests to internal services.

Main Endpoints

```
GET /health
```

Checks application health.

POST /documents/upload

Flow:

```
graph TD; Upload --> Save; Save --> Load; Load --> Preprocess; Preprocess --> Chunk; Chunk --> Embed; Embed --> FAISS;
```

This endpoint is responsible for making the document searchable.

POST /chat

Flow:

```
graph TD; Question --> RAG_Pipeline[RAG Pipeline]; RAG_Pipeline --> Answer;
```

POST /documents/extract

Flow:

```
graph TD; Document --> Extraction_Service[Extraction Service]; Extraction_Service --> Structured_Result;
```

POST /documents/compare

Flow:

```
Document A
+
Document B
↓
Comparison Service
↓
Comparison Result
```

POST /agent/query

Flow:

```
Question
↓
Document Agent
↓
Tools
↓
Final Answer
```

Importance

This is the main integration layer.

The lower-level modules do the work, while `main.py` exposes that functionality externally.

25. frontend/app.py

Purpose

Implements the Streamlit frontend.

Main Sections

The frontend contains interfaces for:

Upload & Index
Ask Documents
Extract
Compare
Agent

Upload

Allows the user to:

Select document
↓
Send to API
↓
Process/index

Ask Documents

Allows:

Question
↓
/chat
↓
Answer

Extract

Calls:

/documents/extract

and displays structured information.

Compare

Calls:

```
/documents/compare
```

and displays comparison results.

Agent

Calls:

```
/agent/query
```

and provides the agent interface.

Importance

This is the presentation layer.

The frontend itself does not contain the core AI logic.

Instead:

```
Streamlit  
  ↓  
FastAPI  
  ↓  
Services
```

This separation is important.

26. tests/

The tests are separated by responsibility rather than having one massive test file.

27. tests/test_document.py

Purpose

Tests document-processing components.

Covers

- PDF loading
 - Image processing
 - OCR behavior
 - Preprocessing
 - Document representation
-

Why It Matters

If document extraction is broken, everything downstream can fail.

For example:

```
Bad PDF extraction
↓
Bad chunks
↓
Bad embeddings
↓
Bad retrieval
↓
Bad answer
```

Therefore document tests protect the beginning of the pipeline.

28. tests/test_rag.py

Purpose

Tests the RAG subsystem.

Covers

- Chunker
 - Embedding service
 - Vector store
 - Retriever
 - RAG pipeline
-

Importance

This is one of the most important test modules because RAG is the central GenAI capability.

Revision Note

When changing:

```
chunk size
embedding model
vector store
retriever
prompt
```

this test file should be checked.

29. tests/test_services.py

Purpose

Tests service-level business logic.

Covers

```
ExtractionService
ComparisonService
```

Why Separate It?

We want to know whether the business logic works independently of FastAPI.

That gives:

```
Unit tests
```

instead of testing everything only through HTTP.

30. tests/test_agent.py

Purpose

Tests the document agent and tools.

Covers

- Agent initialization
 - Tool registration
 - Tool execution
 - Document context
 - Agent behavior
-

Importance

Agent code can be difficult to debug.

Testing the tools separately helps isolate failures.

31. tests/test_api.py

Purpose

Tests the FastAPI layer.

Covers

- Health
 - Upload
 - Chat
 - Extraction
 - Comparison
 - Agent
-

Importance

This verifies that the modules integrate correctly through the actual API layer.

32. `tests/test_evaluator.py`

Purpose

Tests the evaluation framework itself.

Covers

- Evaluation execution
 - Score calculations
 - Citation scoring
 - Report creation
 - Missing-information questions
-

Important Revision

Citation scoring was specifically corrected so that citation-required questions are properly included in the score.

33. `evaluation/`

The evaluation package is separate from ordinary testing.

Testing asks:

Does the code work?

Evaluation asks:

How well does the AI system answer questions?

This is an important distinction.

34. `evaluation/questions.json`

Purpose

Stores evaluation questions.

Categories

The questions cover areas such as:

```
RAG
Citation
Extraction
Reasoning
Missing Information
Comparison
Cross-document
```

Why This Matters

An LLM application needs more than unit tests.

For example:

```
pytest → code correctness
evaluation → AI quality
```

35. `evaluation/evaluator.py`

Purpose

Evaluates DocuMind responses against the predefined evaluation questions.

Responsibilities

- Load questions
- Run answer function
- Evaluate answers
- Evaluate citations
- Calculate metrics
- Save reports

Citation Logic

A citation-required question should be evaluated for:

```
citation_found
```

and:

```
citation_valid
```

The score should include all applicable citation questions.

Importance

This creates a repeatable way to measure system quality.

36. data/

The data folder contains runtime/project data rather than application logic.

37. data/raw/

Purpose

Stores original documents.

The project dataset was organized into:

```
raw/  
├─ invoices/  
├─ forms/  
├─ contracts/  
└─ synthetic/
```

Important Rule

Raw documents should generally remain unchanged.

They represent the original source material.

38. data/raw/invoices/

Contains invoice/receipt-style documents.

Useful for testing:

- OCR
 - Field extraction
 - Amounts
 - Dates
 - Vendor information
-

39. data/raw/forms/

Contains form-style documents.

Useful for testing:

- OCR
 - Structured extraction
 - Key-value relationships
-

40. data/raw/contracts/

Contains contract/legal-style documents.

Useful for:

- RAG
 - Citations
 - Agent queries
 - Contract comparison
 - Clause retrieval
-

41. data/raw/synthetic/

Contains synthetic documents created for controlled comparison/testing scenarios.

These are useful when you need known differences between documents.

For example:

Document A
Payment = 30 days

Document B
Payment = 45 days

This makes comparison evaluation easier.

42. data/processed/

Purpose

Stores processed document artifacts if the application generates them.

Difference From Raw

```
raw/  
↓  
Original input  
  
processed/  
↓  
Application-generated representation
```

43. data/vector_store/

Purpose

Stores the persistent FAISS vector index.

Lifecycle

```
Document  
↓  
Chunks  
↓  
Embeddings  
↓  
FAISS
```

↓
data/vector_store/

Important Revision Point

If the embedding model changes significantly, the existing vector store may no longer be compatible.

In that case, rebuild the vector store.

This was particularly relevant during the OpenAI → Gemini migration.

44. scripts/

Purpose

Reserved for operational/helper scripts.

Scripts should contain utility operations rather than core business logic.

Examples of suitable future scripts:

```
scripts/  
├─ rebuild_index.py  
├─ ingest_dataset.py  
└─ run_evaluation.py
```

Only add these if they are actually implemented.

45. requirements.txt

Purpose

Defines the Python packages needed by the project.

Dependency Categories

Web API

FastAPI
Uvicorn
python-multipart

Configuration

Pydantic
Pydantic Settings
python-dotenv

GenAI

LangChain
LangChain Core
Gemini integration

Vector Search

FAISS

Documents

PyMuPDF
Pillow
pytesseract

Frontend

Streamlit

Testing

pytest

Importance

This file makes the Python environment reproducible.

46. `.env`

Purpose

Stores local configuration and secrets.

Important Variable

```
GEMINI_API_KEY=your_key
```

Other configuration variables control:

```
LLM
Embeddings
Chunking
Retrieval
Application
```

Critical Rule

Never commit `.env`.

Use:

```
.env.example
```

for public configuration templates.

47. `.gitignore`

Purpose

Prevents unnecessary or sensitive files from entering Git.

Important Entries

At minimum:

```
.env
.venv/
__pycache__/
*.pyc
```

Generated vector stores/data should also be considered depending on whether they are intended to be distributed with the project.

48. README.md

Purpose

The README is the **public-facing project introduction**.

The technical documentation you are reading now is different.

README

Answers:

"What is this project and how do I run it?"

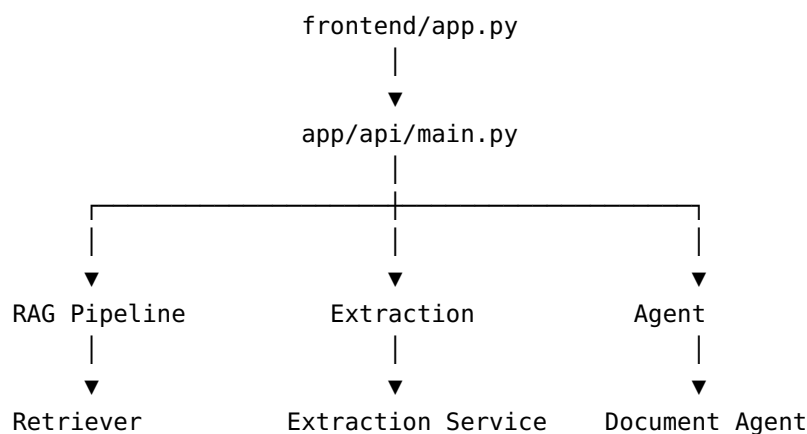
This document

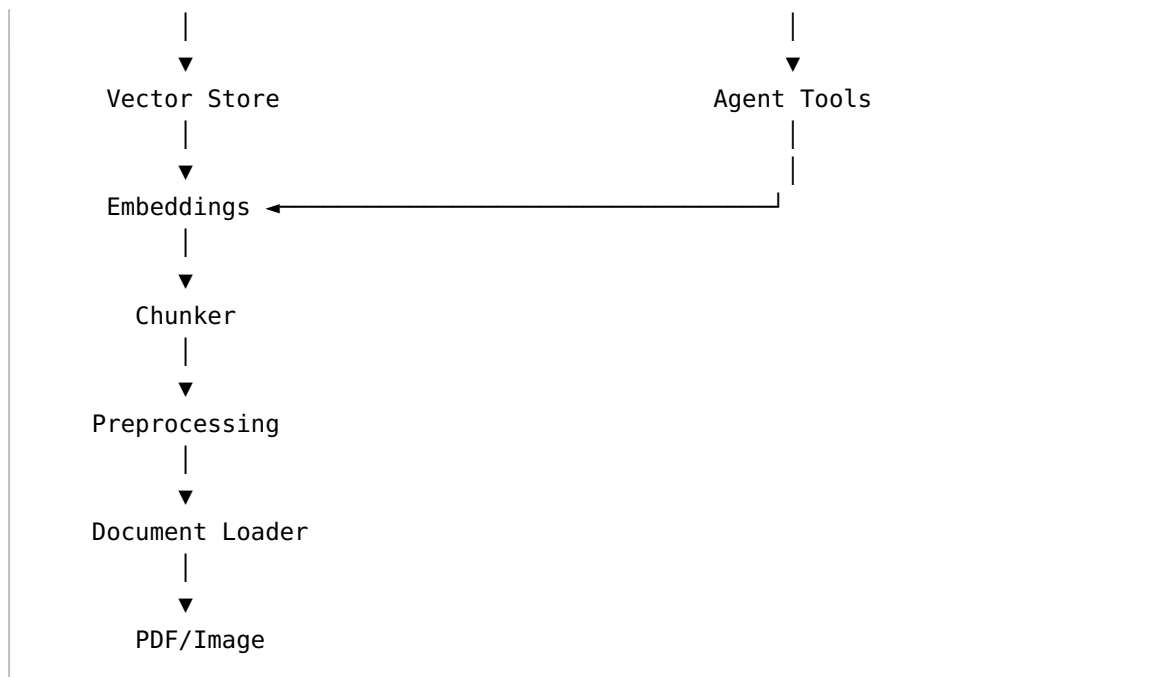
Answers:

"How does every part of this project work and how do I maintain it?"

49. Complete Dependency Graph

A useful way to remember the project is to follow dependencies from bottom to top.





50. The Most Important Files to Study

You do not need to memorize every file equally.

For a technical interview, focus on these files first:

Level 1 — Core GenAI

```
app/rag/embeddings.py
app/rag/retriever.py
app/rag/pipeline.py
app/agents/tools.py
app/agents/document_agent.py
```

These demonstrate your GenAI knowledge.

Level 2 — Document Intelligence

```
app/document/loader.py
app/document/ocr.py
app/document/preprocessing.py
app/rag/chunker.py
```

These explain how raw documents become searchable information.

Level 3 — Business Logic

```
app/services/extraction_service.py
app/services/comparison_service.py
```

These demonstrate that the project is more than a basic chatbot.

Level 4 — Engineering

```
app/api/main.py
app/models/schemas.py
app/core/config.py
app/core/logger.py
```

These demonstrate application architecture.

Level 5 — Quality

```
tests/
evaluation/
```

These demonstrate testing and AI evaluation.

51. Most Important Data Flow to Memorize

If an interviewer asks:

"Explain how your document Q&A system works."

Your mental answer should be:

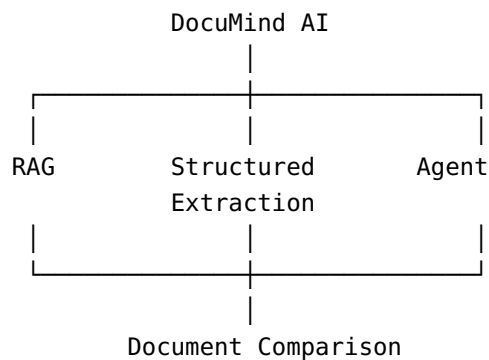
1. User uploads a document.
2. FastAPI receives the file.
3. The document loader extracts pages.
4. OCR is used where required.
5. Text is conservatively preprocessed.

6. Each page is split into chunks.
7. Chunk metadata preserves document/page information.
8. Gemini converts chunks into embeddings.
9. FAISS stores those vectors.
10. User asks a question.
11. The question is embedded.
12. FAISS retrieves the most relevant chunks.
13. The retrieved chunks are inserted into a grounded prompt.
14. Gemini generates the answer.
15. Source/page metadata is returned with the answer.

That is the **heart of DocuMind AI**.

52. Why the Project Is More Than a PDF Chatbot

The project contains several distinct GenAI patterns.



RAG

Answers questions from retrieved document context.

Structured Extraction

Converts document content into structured fields.

Agent

Allows an LLM to choose controlled tools.

Comparison

Uses extraction + deterministic normalization/comparison.

These four capabilities make the project significantly stronger as a portfolio project.

53. When You Modify the Project

Use this troubleshooting map.

Upload fails

Check:

```
main.py
loader.py
preprocessing.py
chunker.py
embeddings.py
vector_store.py
```

Embedding error

Check:

```
config.py
embeddings.py
GEMINI_API_KEY
EMBEDDING_MODEL
```

Retrieval is poor

Check:

```
chunker.py
embeddings.py
retriever.py
```

Especially:

```
CHUNK_SIZE
CHUNK_OVERLAP
TOP_K
```

Answer is hallucinating

Check:

```
pipeline.py
retriever.py
```

Verify:

- Relevant chunks are retrieved.
- Context is correctly constructed.
- Grounding instructions are present.

Citations are wrong

Check:

```
loader.py
chunker.py
pipeline.py
schemas.py
```

The most likely issue is lost page metadata.

Extraction is incorrect

Check:

```
extraction_service.py
schemas.py
```

Agent fails

Check:

```
document_agent.py
tools.py
```

Then verify the underlying RAG/extraction services independently.

Comparison is wrong

Check:

```
comparison_service.py
```

especially:

```
normalization
synonym mapping
numeric handling
```

Frontend fails

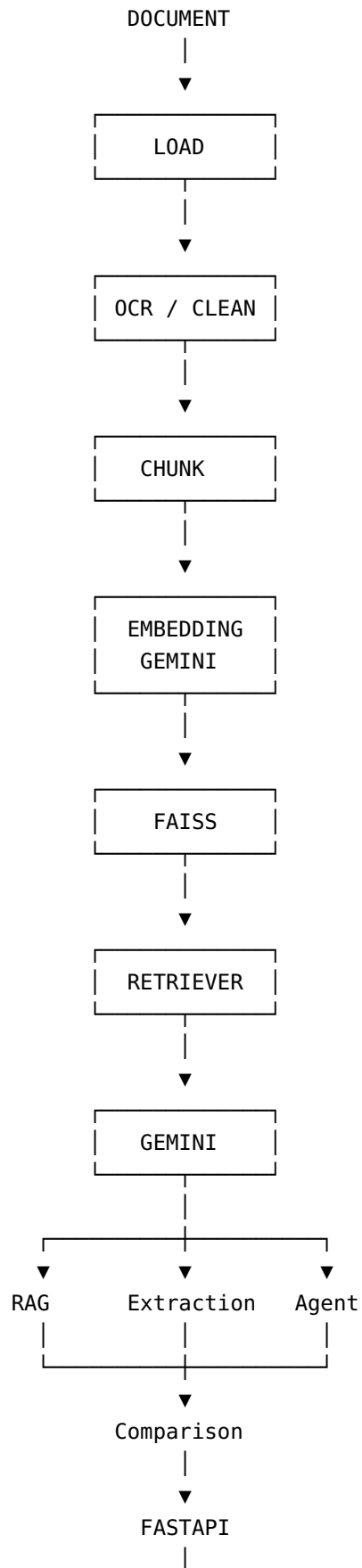
Check:

```
frontend/app.py
main.py
schemas.py
```

The frontend is dependent on the API response structure.

54. Final Mental Model

The simplest way to remember the entire project is:



55. One-Sentence Responsibility of Every Major File

For quick revision:

File	Remember It As
<code>config.py</code>	Central configuration
<code>logger.py</code>	Central logging
<code>schemas.py</code>	Shared data contracts
<code>loader.py</code>	Raw document → structured document
<code>ocr.py</code>	Image → text
<code>preprocessing.py</code>	Clean document text
<code>chunker.py</code>	Document → retrieval chunks
<code>embeddings.py</code>	Text → vectors
<code>vector_store.py</code>	Store/search vectors
<code>retriever.py</code>	Find relevant chunks
<code>pipeline.py</code>	RAG question answering
<code>tools.py</code>	Agent capabilities
<code>document_agent.py</code>	LLM tool-calling
<code>extraction_service.py</code>	Document → structured fields
<code>comparison_service.py</code>	Document A vs Document B
<code>main.py</code>	FastAPI integration
<code>frontend/app.py</code>	User interface
<code>test_document.py</code>	Document tests
<code>test_rag.py</code>	RAG tests
<code>test_services.py</code>	Service tests
<code>test_agent.py</code>	Agent tests
<code>test_api.py</code>	API tests
<code>test_evaluator.py</code>	Evaluation tests

File	Remember It As
<code>questions.json</code>	Evaluation dataset
<code>evaluator.py</code>	AI quality measurement
<code>requirements.txt</code>	Python dependencies
<code>.env</code>	Secrets/configuration
<code>.gitignore</code>	Files Git should ignore
<code>README.md</code>	Public project documentation

56. Final Architecture Summary

DocuMind AI can ultimately be understood as **three major layers**.

Layer 1 — Document Intelligence

Loader
OCR
Preprocessing
Chunking

Purpose:

Convert messy real-world documents into clean, structured, searchable information.

Layer 2 — GenAI Intelligence

Embeddings
FAISS
Retriever
RAG
Extraction
Agent
Comparison

Purpose:

Understand, retrieve, extract, reason over, and compare document information.

Layer 3 — Application Layer

```
FastAPI
Streamlit
Configuration
Logging
Testing
Evaluation
```

Purpose:

Turn the GenAI capabilities into a usable and maintainable application.

57. The Core Engineering Lesson

The most important thing to remember about this project is that **the LLM is only one component**.

The actual system is:

```
Documents
+
Data Processing
+
Metadata
+
Chunking
+
Embeddings
+
Vector Search
+
Retrieval
+
Prompting
+
LLM
+
Structured Output
+
Tools
+
Business Logic
+
API
+
UI
```

+
Testing

That is what makes DocuMind AI a proper **GenAI engineering project** rather than simply an LLM wrapper.